domain ; therefore
one off point
of two on
for the domain
e to the

routine is
est points in
- testing
utine in

tical white
this without
riable at

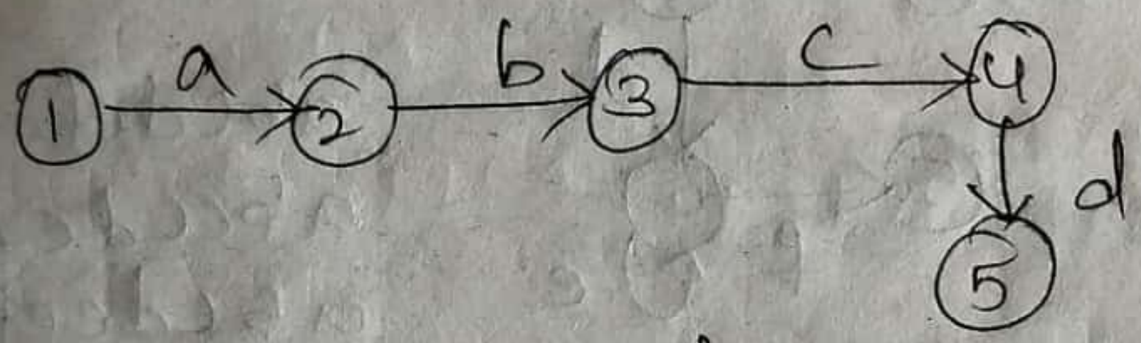
Date
30/11/26

UNIT-III

Paths, Path Products and Regular Expression

→ The path is a combination of several path segments. Each segment is given as a name.

→ The path is expressed as concatenation of all path segments on that path.



The path products :- The path product means concatenation of all the paths

→ In the figure there are four path segments a, b, c, d. The path from node 1 to 5 can be expressed as concatenation.

→ That is expressed as abcd

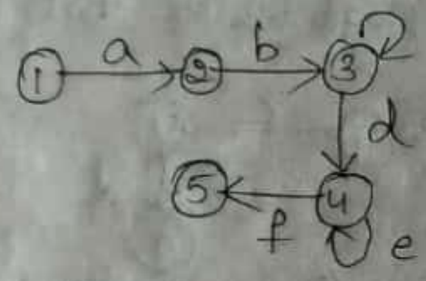
→ This abcd is known as the path product.

→ If A, B are the path segments where A = abc B = def then the path product is AB = abc* def the final equation is abc def.

For Ex:- A and B set of paths

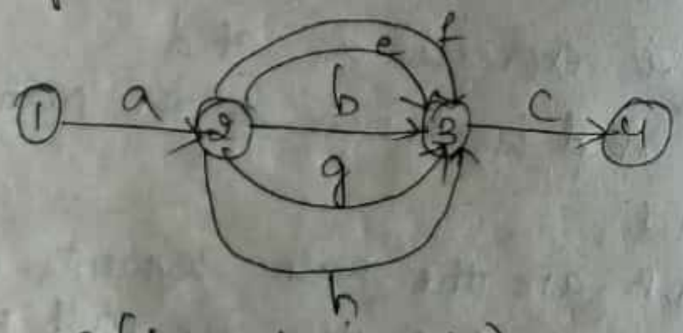
represented as A = ab + cde where B = pqr + st then the path product A,B. is abpqr + abst + cdepqr + cd

Path Expression:- The expression which represents the set of all posible paths between the entry to exit nodes there is known as path expression



abcdef
abccdef
abcdeef

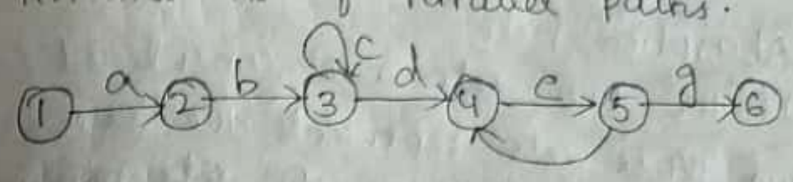
The path sums:- The paralled path segments in this path indicates the flow can occur in any one of those parallel paths. It can be represented by a plus (+).



a(b+e++f+g+h)c

abc + aec + afc + agc + ahc.

Loops:- It is said to be a loop when the same link is represented infinite no. of times or it can be considered as infinite no. of parallel paths.

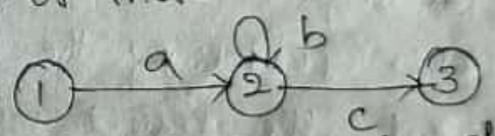


In the above figure self loop is at node 3, the processing of the loop which retains the same state is known as self loop.

→ The loop on only 1 node & itself is known as self loop

→ It can be represented closure (*)

→ a+ denotes occurence of an element one or more times (not zero)



Another example of path loops.

Distributive Law

→ The combination of product and sum operations are said to be distributive

→ The normal multiplication rules can be applied to these laws.

Example:- a(b+c+d+e+f)g

abg + acg + adg + aeg + afg

This expression is all possible paths from node 1 to node 4.

$A+B = A+A = A$

Absorption rule

If the two sets are considered (say) and those sets represents the same set of paths, then union of sets remains same.

As $A=B$,

If A is considered as the set of paths and $A = p + pq + pqr$ then,

According to Absorption Rule.

$A+p = A+pq = A+pqr = A$

Identity elements

In general, identity element is the one which after ever performing the operation with that element, the result would be same where

a is operand
op is operator
e is identity element

If a path segment or link is ~~separat~~ represented as a^0, then the length of the path is zero.

Date 31/1/26

Node Reduction procedure Algorithm

The path product is Associative.

Rule 1: $A(BC) = (AB)C = ABC$.

where A, B, C are path names, set of path names or path expressions.

The path is a set union operation, it is clearly commutative and Associative.

RULE 2: $X+Y = Y+X$

RULE 3: $(X+Y)+Z = X+(Y+Z) = X+Y+Z$

Distributive laws:-

The product and sum operations are distributive, and the ordinary rules of multiplication apply; that is

RULE 4: $A(B+C) = AB+AC$ and $(B+C)D = BD+CD$

Applying these rules to the below figure 5.1a yields

$e(a+b)(c+d)f = e(ac+ad+bc+bd)f$

$= eacf + eadf + ebcf + ebdf$

Absorption rule:-

If X and Y denote the same set of paths, then the union of these sets is unchanged; consequently.

RULE 5: $X+X = X$ (Absorption Rule)

If a set consists of paths names

and a member of that set is added to it, the "new" name, which is already in that set of names, contributes nothing and can be ignored.

For example.

If $x = a + aa + abc + abcd + def$ then

$x + a = x + aa = x + abc = x + abcd$

$= x + def = X$

It follows that any arbitary sum of identical path expressesions reduces to the same path expression.

RULES 6–16

The following rules can be derived from the previous rules:

RULE 6: $x^n + x^m = x^n$ if $n > m$

RULE 6: $x^n + x^m = x^m$ if $m > n$

RULE 7: $x^n x^m = x^{n+m}$

RULE 8: $x^n x^* = x^* x^n = x^*$

RULE 9: $x^n x^+ = x^+ x^n = x^+$

RULE 10: $x^* x^+ = x^+ x^* = x^+$

RULE 11: $1 + 1 = 1$

RULE 12: $1x = x1 = x$

Following or preceding a set of paths by a path of zero length does not change the set.

RULE 13: $1^n = 1^0 = 1^* = 1^+ = 1$

No matter how often you traverse a path of zero length, It is a path of zero length.

RULE 14:- $1^+ + 1 = 1^* = 1$

The null set of paths is denoted by the numeral 0. it obeys the following rules

RULE 15: $x + 0 = 0 + x = x$

RULE 16: $0x = x0 = 0$

Reduction procedure:-

Reduction procedure Algorithm

This section ~~sectio~~ represents a reduction procedure for converting a flow graph whose links are labeled with names into a path expression that denotes the set of all entry/exit paths in that flow graph. The procedure is a node-by node removal algorithm.

The steps in Reduction algorithm are as follows:-

1. Combine all serial links by multiplying their path expressions.

2. Combine all parallel links by adding their path expressions.
3. Remove all self-loops (from any node to itself) by replacing them with a link of the form x^*, where x is the path expression of the link in that loop.

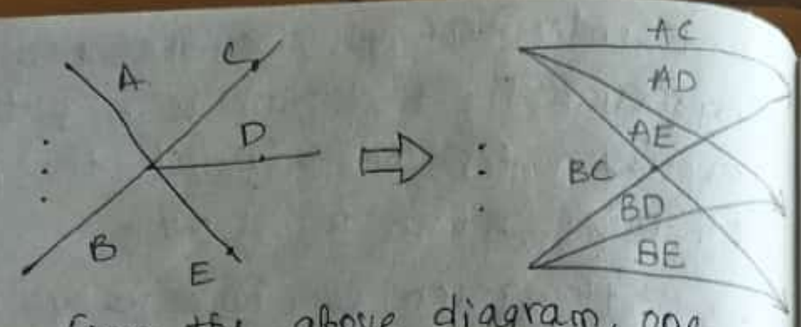
STEPS 4-8 ARE IN THE ALGORITHM LOOP:

4) Select any node for removal other than the initial or final node. Replace it with a set of equivalent links whose path expressions corresponds to all the ways you can form a product of the set of inlinks with the set of outlinks of that node.
5) Combine any remaining serial links by multiplying their path expressions.
6) ~~Remove~~ Combine all parallel links by adding their path expressions.
7) Remove all self-loops as in step 3.
8) Does the graph consists of a single link between the entry node and the exit node? If yes, then the path expression for that link is a path expression for the original flow graph; otherwise, return to step 4.

→ A flowgraph can have many equivalent path expressions between a given pair of nodes; that is, there are many different ways to generate the set of all paths between two nodes without affecting the content of that set.
→ The appearence of the path expression depends, in general, on the order in which nodes are removed.

CROSS-TERM STEP (STEP 4):

The cross-term step is the fundamental step of the reduction algorithm.
→ It removes a node, thereby reducing the number of nodes by one.
→ Successive applications of this step eventually get you down to one entry and one exit node. The following diagram shows the situation at an arbitary node that has been selected for removal.

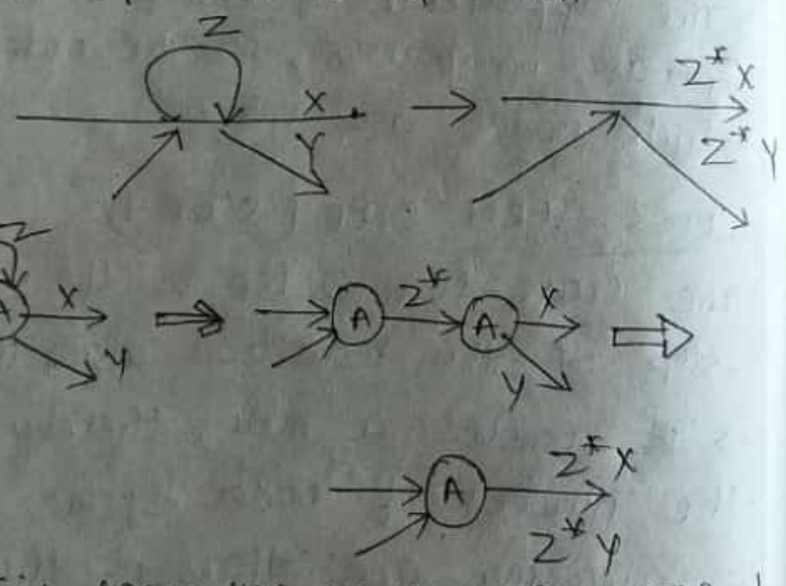


From the above diagram, one can infer:

$(a+b)(c+d+e) = ac+ad+ae+bc+bd+be$.

Loop removal operations

There are two ways for looking at the loop-removal operation



In this way, we remove the self-loop and then multiply all outgoing link z^*

* In the second way, we split the node into two equivalent nodes, call them A' and put in a link between them whose path expression is z^*. Then we remove node 'A' using steps 4 and 5 to yield outgoing links whose path expressions are z^*x and z^*y.

31/1/26

A Reduction procedure - Example :-

Let us see by applying this algorithm to the following graph where we remove several nodes in order ; that is

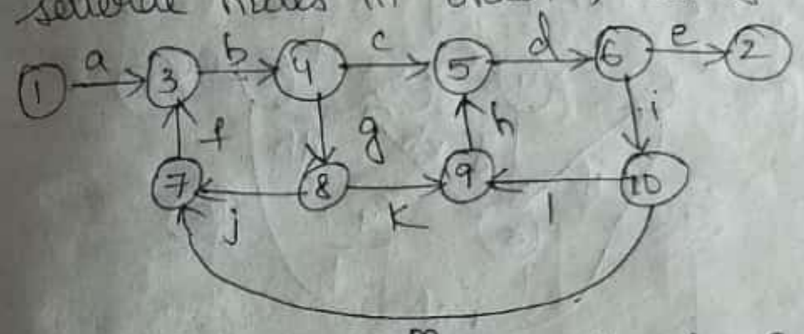
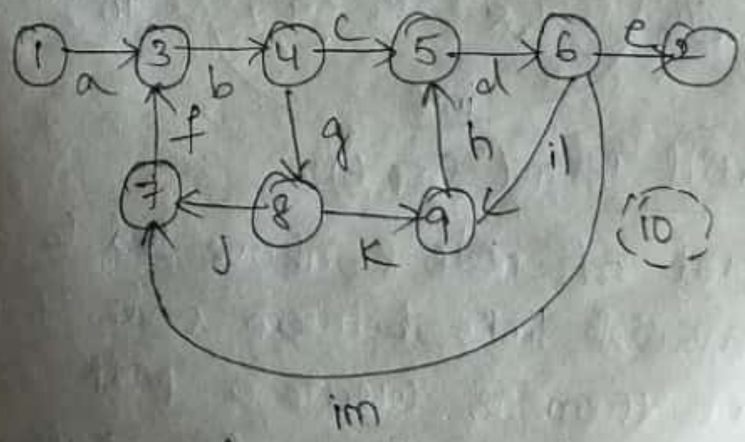


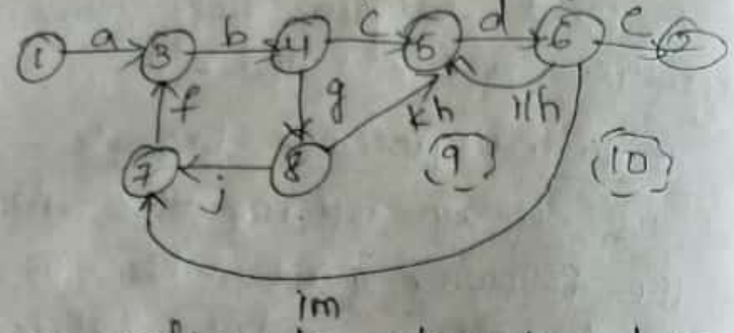
Figure 5.5: Example flow graph for demonstrating reduction procedure.

Remove node 10 by applying step 4 and combine by step 5 to yield

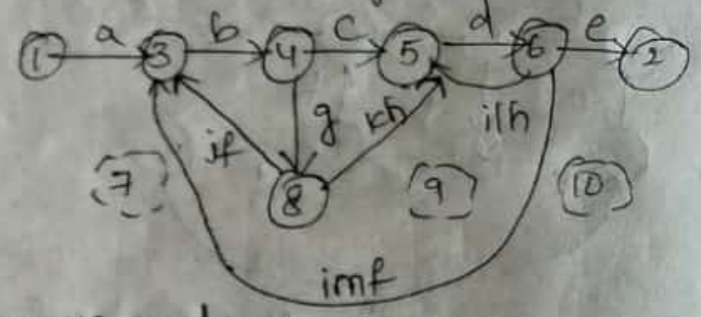


* Remove node 9 by applying step 4 and 5 to yield.

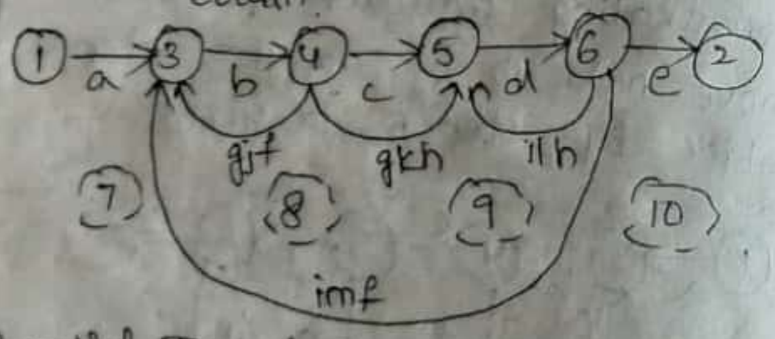
Remove node 9 by applying step 4 &



Remove node 7 by steps 4 and 5 follow



Remove node 8 by steps 4 & 5, to obtain:

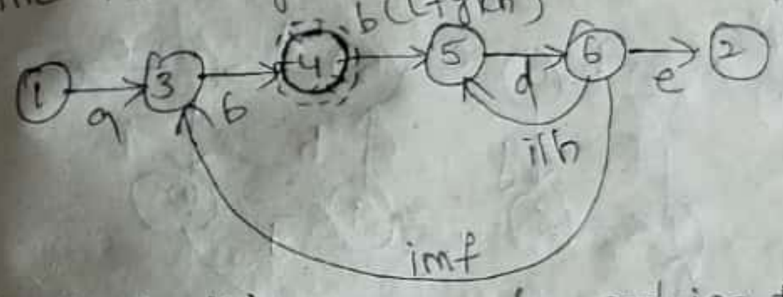


Parallel Term (step 6):-

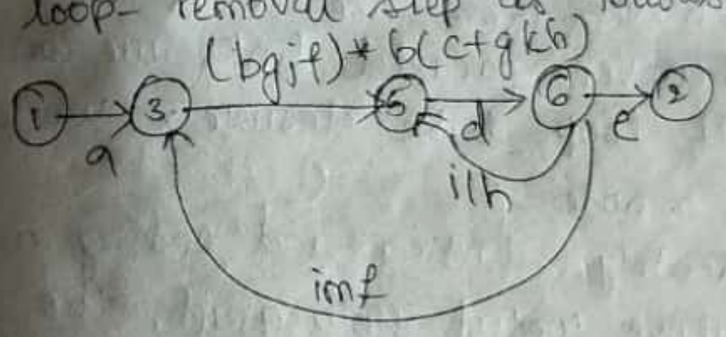
Removal of node 8 above led to a pair of parallel links between node 4 and 5. combine them to create a path expression for an equivalent link whose path expression is c+gkh; that is

Loop term (step 7):-

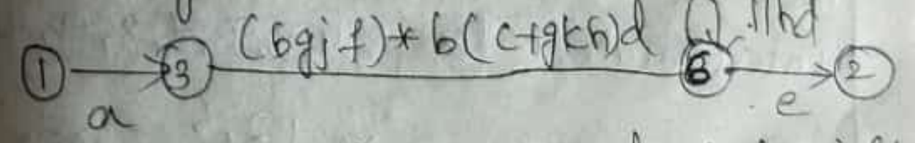
Removing node 4 leads to a loop term. The graph has now been replaced with the following equivalent simpler graph



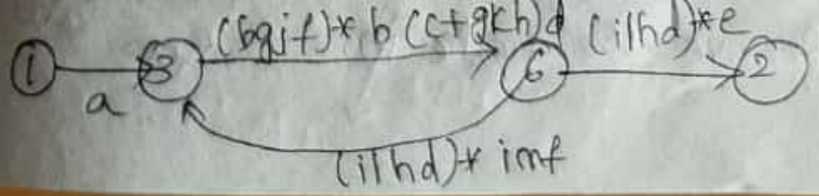
continue this process by applying the loop-removal step as follows:



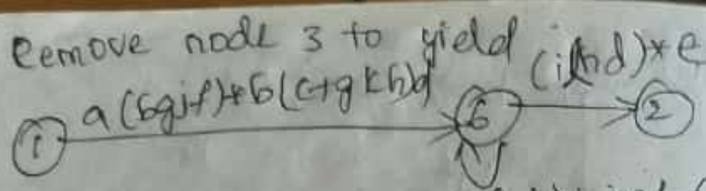
Removing node 5 produces:



Remove the loop at node 6 to yield



Remove node 3 to yield (i|hd)*e



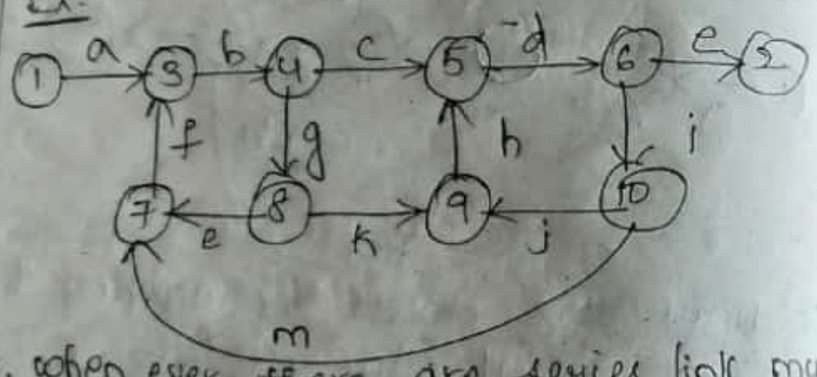
(i|hd)* imf (bgjf) * b(c+gkh)d

Removing the loop and then node 6 result in the following expression

a(bgjf)* b(c+gkh)d (i|hd)* imf (bgjf) * b(c+gkh)d) * (i|hd)*e.

3/11/26

Ex:-



1. when ever there are series link multiply
2. Whenever parallel links we can add.
3. self loop take closure *, where x is a node.
4. Identify starting and ending nodes the above control flow graph, the start node 1, end node 2

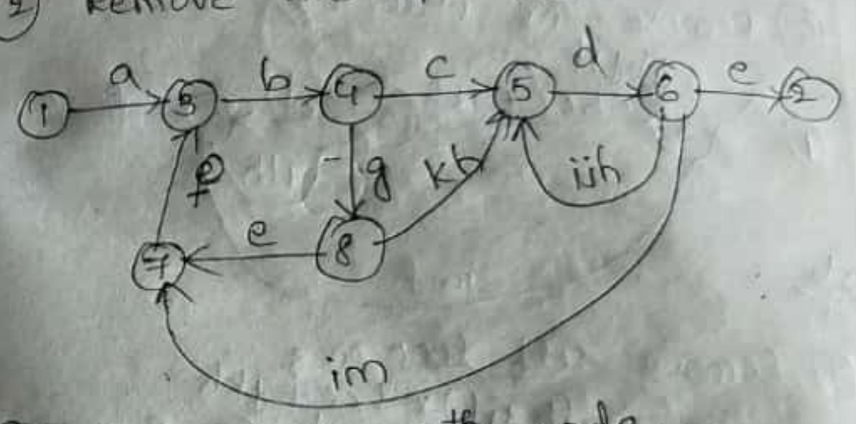
Intermediate nodes 3, 4, 5, 6, 7, 8, 9, 10.

Steps

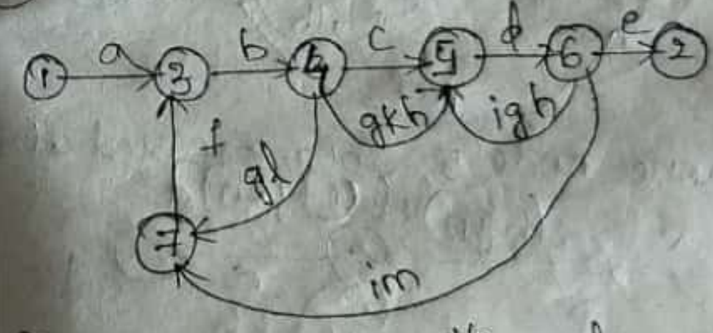
Rule 1,2,3 again same from above.

(1) Remove the node 10.

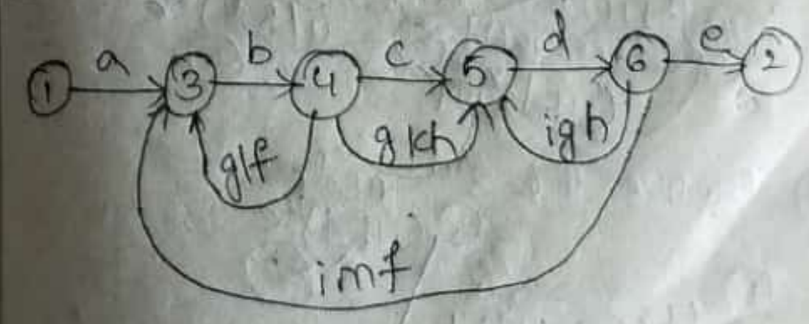
(2) Remove the 9th node



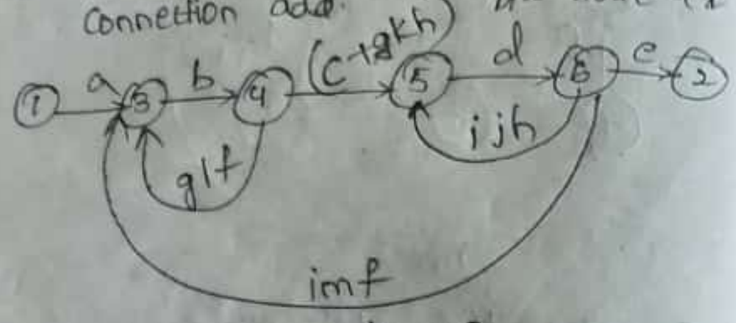
(3) Remove the 8th node.



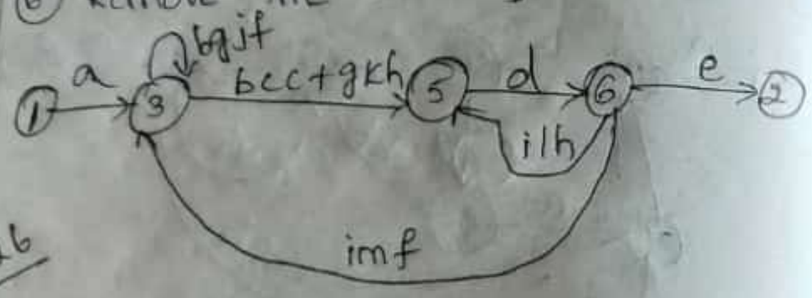
(4) Remove the 7th node



⑤ Whenever the node having a parallel connection add. Here node 4 & 5

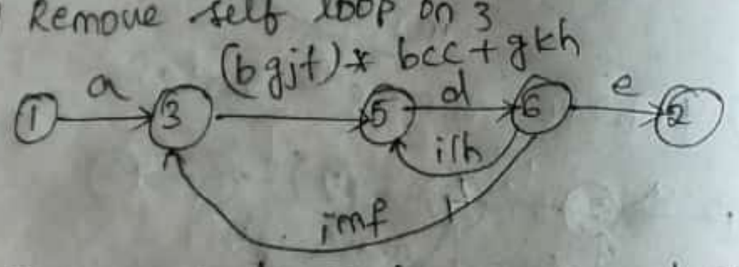


⑥ Remove the node 4

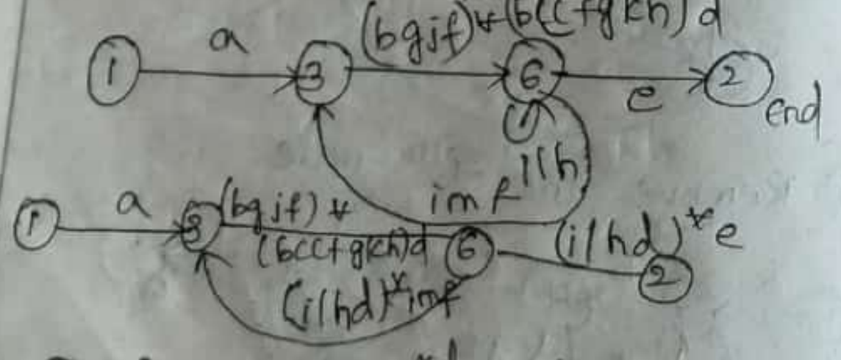


2/2/26

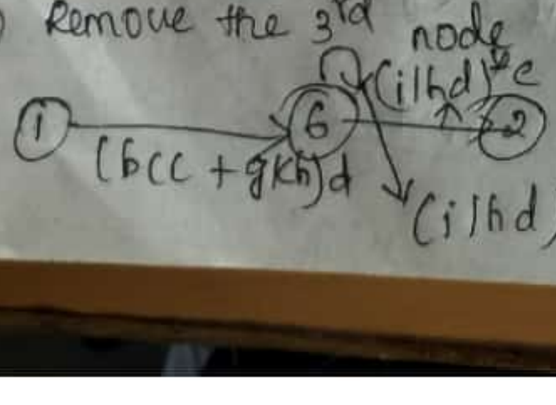
⑦ Remove self loop on 3



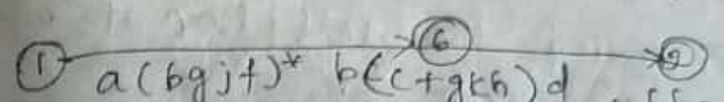
⑧ Remove node 5, loop occured at 6



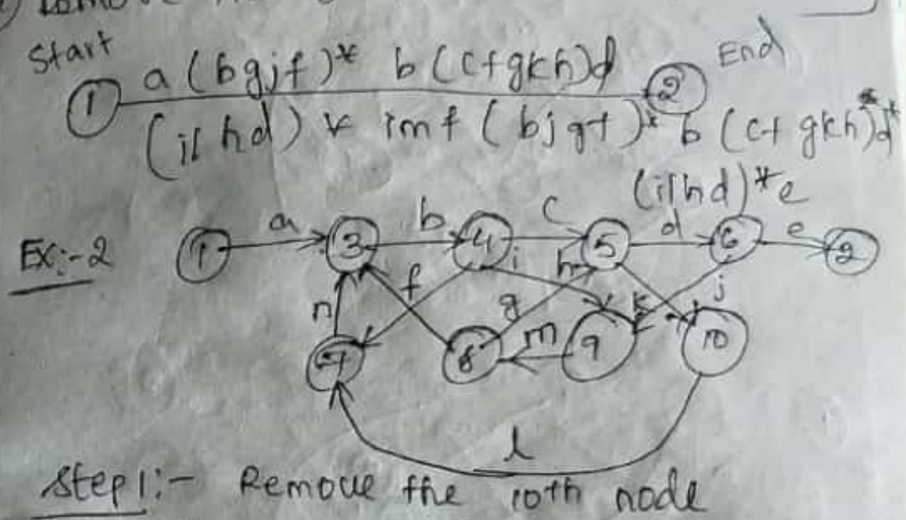
⑨ Remove the 3rd node



⑩ Remove self loop at node 6.



⑪ Remove the 6th node



Ex:-2

Step 1:- Remove the 10th node

Step 2:- Remove 9th node

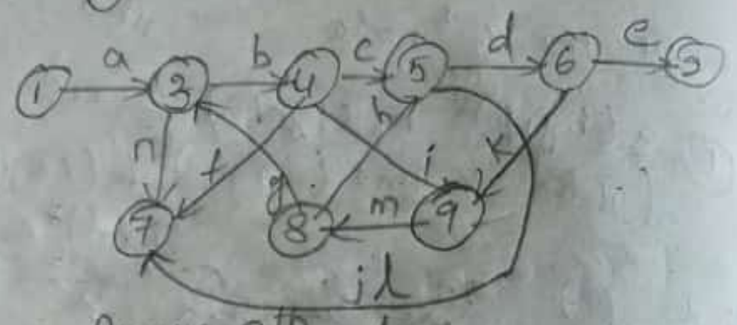
3. Remove, 8th & 7th after removing if there are several links so we can combine,
6. There is loops b/w 5 & 6
7. Remove node 4. Can be eliminated and it results in term
8. There are some parallel connection b/w 3rd & 6th node.
9. There is a self loop, remove self loop.
10. Remove the 5th node.
11. Remove the node 3 which leads self

loop at node 6.

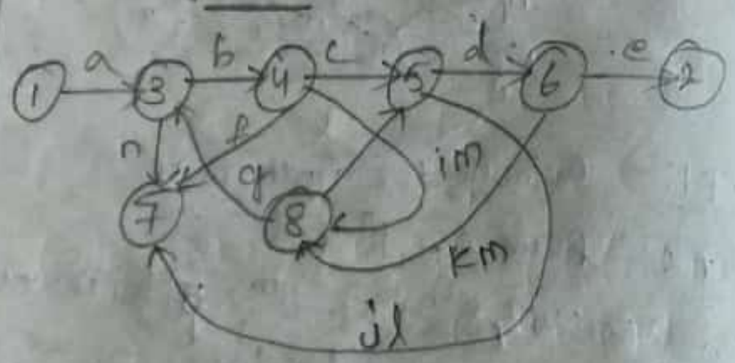
ii. Removing the self loop at node 6.

b. remove node 6.

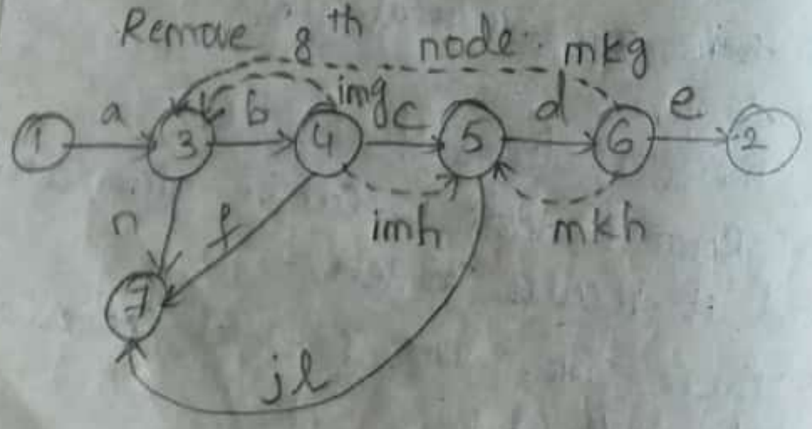
(i) Remove the 10th node.



Remove 9th node



Remove 8th node mkg

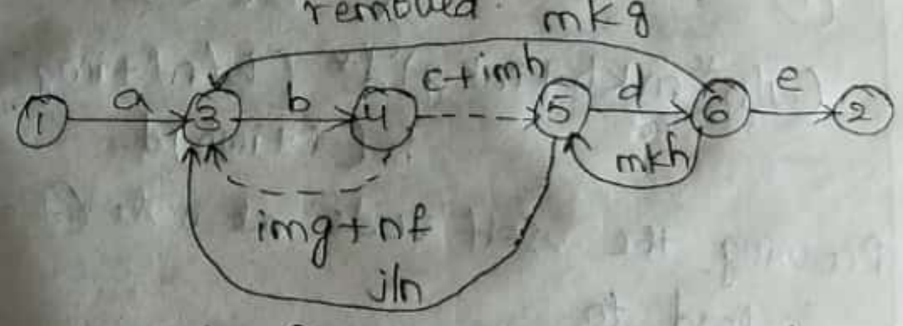


Remove node (4)

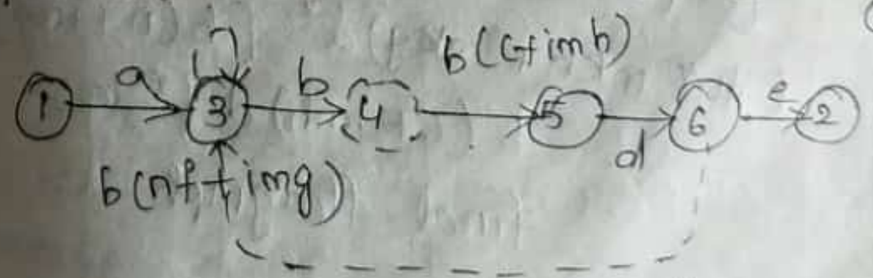
There are several parallel links. So combining them by adding them.



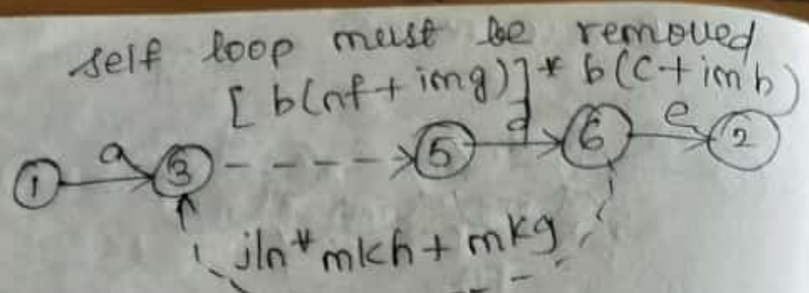
Loops between node 5 and 6 are removed



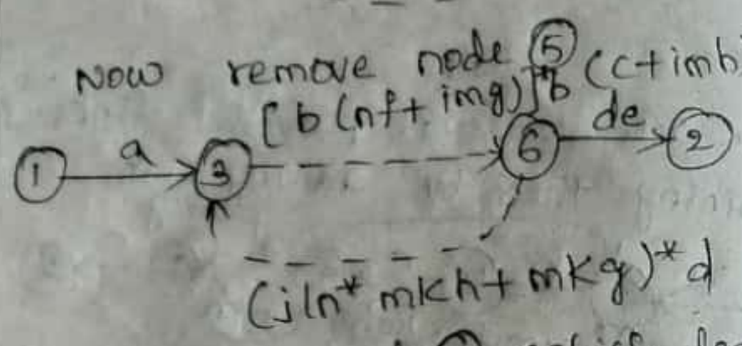
Now node (4) can be eliminated and it results in a term and combine the parallel links between node (3) & node (6)



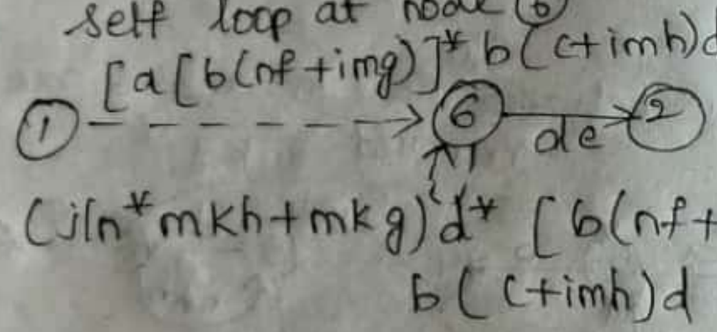
self loop must be removed



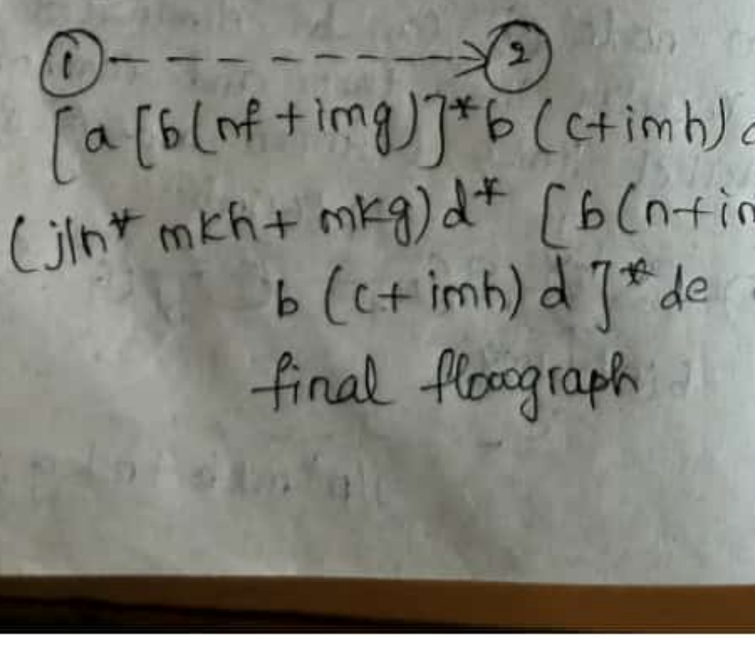
Now remove node (5)



Now, remove node (3) which leads to self loop at node (6)



Removing the self node at node (6) will lead to



final floograph

Date 3/2/26

Applications

The node reduction removal algorithm have several applications and for every application a common pattern or common procedure be followed

Common patterns

→ Each and every graph should be converted to path expression.

→ The set of Arithmetic rules must be derived for the selected property of the paths

→ The link names should be replaced by link weights

→ so that the path expression will be converted into some algebraic expression

The list of applications include as follows

1. Maximum path count Arithmetic
2. structured code.
3. Lower path count Arithmetic
4. Probability of being at a certain point.
5. Mean processing time of a routine.
6. Push/pop, Get/Return problem solving.

Maximum path count Arithmetic

→ The most important application of node removal algorithm is to find the maximum number of paths in the graph.

→ For each and every link in the graph has to be assigned a weight.
→ And at loops, it has to be numbered with the digit which gives the maximum number of times the loop can be taken.
→ If the count of the loop is infinite then the maximum path count would be infinite and the process can be stopped.

Arithmetic Rules

The three Arithmetic rules that are to be followed in reduction are:
→ Parallel rule
→ series rule.
→ Loop rule.

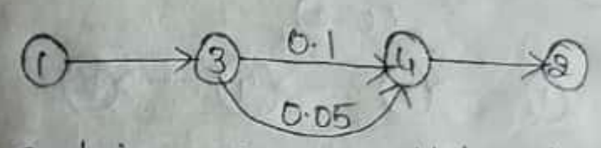
The values that should be considered in case of series, parallel and loop cases are:

	Parallel	Series	Loop
Path Expression	A+B	AB	A^n
Weight Expression	W_A+W_B	$W_A \cdot W_B$	$\sum_{i=0}^{n} W_A$

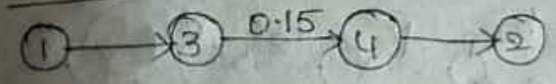
parallel rule

→ The parallel rule is explained as the term in first expression is summed up with each term in the second expression.
→ If the weights of links are w_a and w_b then result is $w_a + w_b$

Ex:-



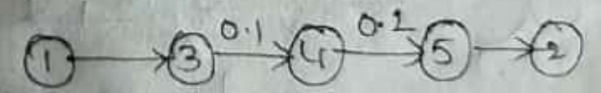
After applying the parallel rules as follows:-



Series Rule

→ Each term in path expression is concatenated or multiplied with each term in second path expression.
→ If w_a and w_b are weights then the result is $w_a \cdot w_b$.

Ex:-



The links between the nodes 3,4,5 are serial and link weights are 0.1 and 0.2 the resultant of these two are

$0.1 * 0.2 = 0.02$

Loop rule:- The loop rule is considering the number of times the path is repeated.



$\sum_{i=0}^{n} (W_A)$

$= \sum_{i=0}^{2} (1*1)$

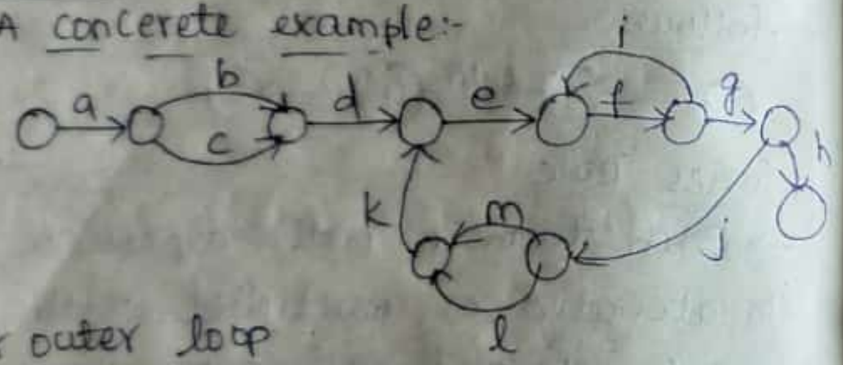
$= 1^0 + 1^1 + 1^2 = 1+1+1 = 3$





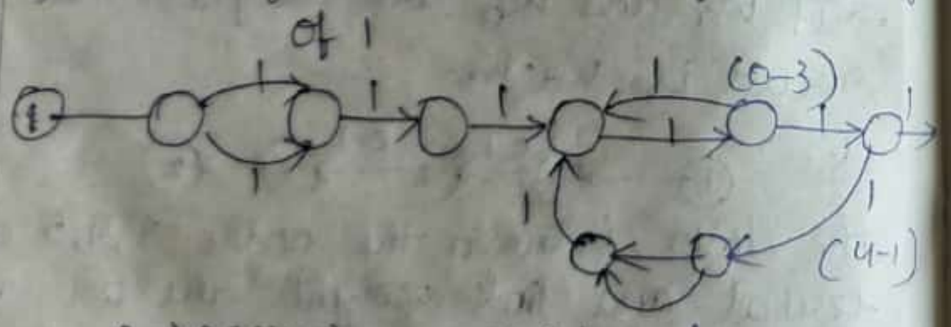
Example

A concerete example:-



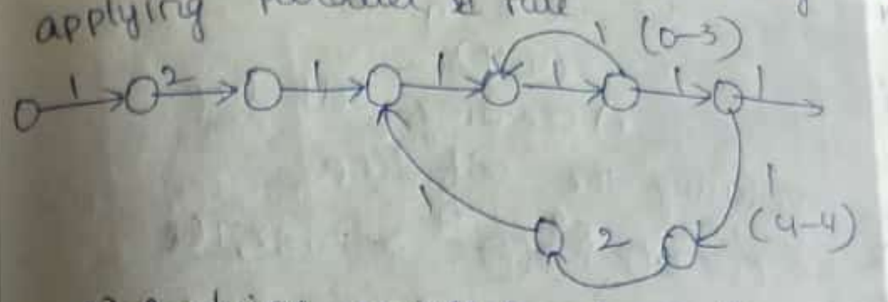
For outer loop

1) All single links are assigned weight of 1



(a) Assigning weights to each link

2) All parallel loops are reduced by applying parallel rule



b) Applying parallel rule to flowgraph

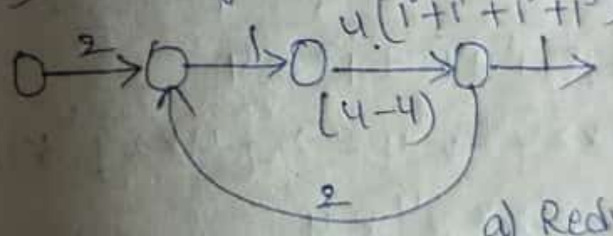
3) Serial links are combined by applying series rule



c) Applying series rule to flowgraph

For inner loop

1) Reducing the inner loop



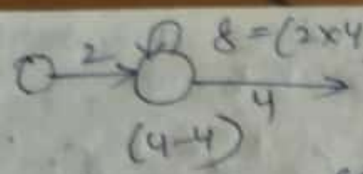
a) Reducing inner loop

2) combining serial links



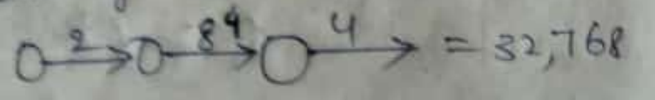
b) Combining serial links

Applying cross-term step to create self loop



c) creating self loop.

Reducing the self loop



d) Reduced Flowgraph.

The alternate way to calculate number of paths is by substituting corresponding weight of each link in the path expression which is.

$a(b+c)d\{e(fi)^* fgj(m+1)k\}^* e(fi)^* fgh$

$= 1(1+1)1\{1(1\times1)^3 1\times1\times1(1+1)1\}^4 1(1\times1)^3 1\times1\times1$

$= 1(2)\{1(1)^3 \times(2)\}^4 * 1^3$

$1^3 = 1^0+1^1+1^2+1^3 = 4$

$2\{1(4)\times2\}^4 * 4 = 2\{4\times2\}^4 * 4$

$= 2 * 8^4 * 4$

$= 32,768$

Structured code

→ A structured code is defined as the code which does not have any loops or any branches.

→ A node by node procedure is used to test the structured code.

→ If the result expression is very large and ugly, it is known as unstructured and if it simple, it is known as structured

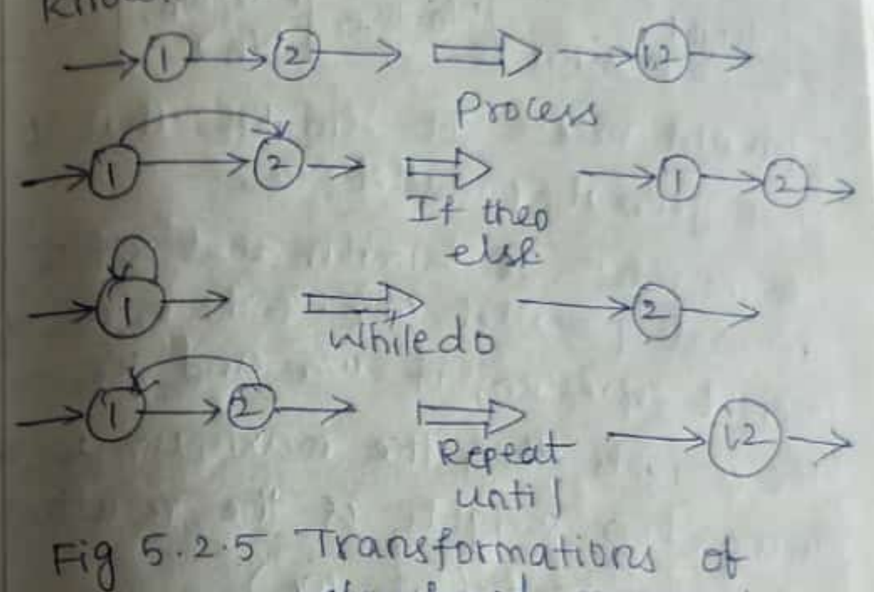


Fig 5.2.5 Transformations of structured flowgraph.

Lower path count Arithmetic

→ The lower path approximated cannot be a true lower count because there would be some unachievable paths in the following which reduces the actual number of paths in the flow graph

→ The Arithmetic rules followed for finding the lower path count are listed as in the table.

Table 5.2 Arithmetic Rules

	Parallel	series	Loop
Path Expression	A+B	AB	A^n
Weight Expression	W_A-W_B	Max(W_A, W_B)	1, W_1

parallel rule:- It similar that of a general parallel case.

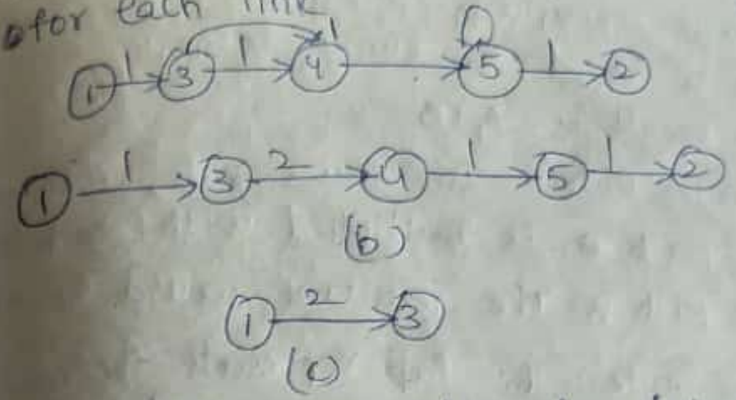
series rule:- It similar that of a general parallel case. But in case of weight expression, if w_a and w_b are the weights then the maximum weight among two weights is the resultant value i.e., $max(w_a, w_b)$

Loop rule:- The essence of loop rule is that in lower path count Arithmetic the loop is considered only once. And the path count should be multiplied with loop weight.



The path expression for this flow graph is $a(b+e)c^*fd$

The minimum number of paths can be determined by substituting the weights for each link



This value can also be determined by substiting the value in the path expression ie., $a(b+e)cfd \Rightarrow 1(1+1)*1*1 = 2$

Date 4/2/26

Push/Pop, Get/Return

The pair of operations of PUSH/POP are generally used to

PUSH - Insert elements into stack

POP - Remove elements from stack.

The other complimentary operations apart from PUSH/POP are

Get/Return

open/close Start/Stop.

Table 5.5 Arithmetic Rules

	Parallel	Series	loop
Path Expression	A+B	AB	A^*
Weight Expression	W_A+W_B	W_A, W_B	W_A^*

Parallel rule:- If there are parallel path expressions, every term in expression A is added to every term in expression B. Result is A+B

Series Rule:- In series rule, each term in path A is multiplied with each term in path B. The resultant would be AB.

Loop rule:- The loop represents the number of times the path is represented. The resultant would be A^*. * represents the number of iterations a loop is repeated.

Table 5.6 Multiplication Table for PUSH/POP

*	P_1	P_2	1
P_1	P_1^2	1	P_1
P_2	1	P_2^2	P_2
1	P_1	P_2	1

In the above table,

- P_1 represents PUSH operation
- P_2 represents POP operation.
- 1 represents no operation
- When two push operations are performed, the result would be P_1^2 as both elements are inserted and stack pointer increases by 2.

→ The same way with two simultaneous pop operations. The stack pointer would be decreased by 2 as two elements are removed from the stack.

→ The simultaneous push and pop or vice versa would result in no operation

Table 5.7 Addition Table for PUSH/POP

+	P_1	P_2	1
P_1	P_1	P_1+P_2	P_1+1
P_2	P_1+P_2	P_2	P_2+1
1	P_1+1	P_2+1	1

The same analysis works for addition which is shown in above table.

Example:- Consider the following flowgraph

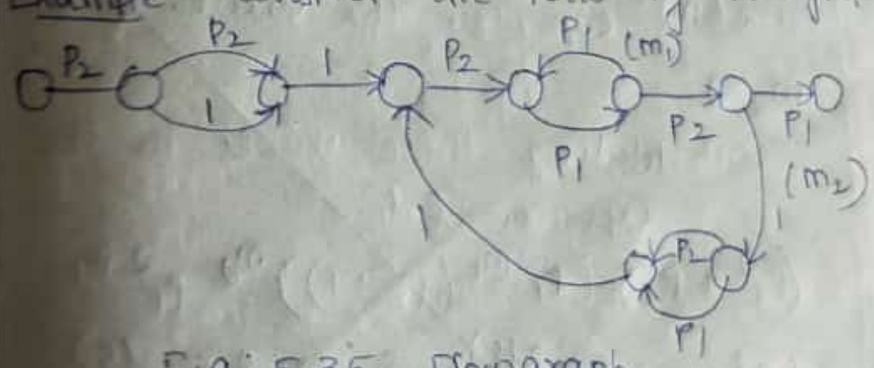


Fig 5.35 Flowgraph.

.path expression for the flowgraph fig 5.35 is,

$$P_2(P_2+1)\,1\{P_2(P_1P_1)^{m_1}P_1P_21(P_1+P_2)1\}^{m_2}$$
$$P_2(P_1P_1)^{m_1}P_1P_2P_1$$

Simplifing the above expression,

$(P_2^2+P_2)\{(P_2(P_1P_1)^{m_1}(P_1+P_2)\}^{m_2}(P_1P_1)^{m_1}$ [$\because P_1P_2=1$]

$(P_2^2+P_2)(P_1^{2m_1}(P_2^2+P_1P_2)\}^{m_2}P_1^{2m_1}$

$(P_2^2+P_2)(P_1^{2m_1}(P_2^2+1)\}^{m_2}P_1^{2m_1}$ [$\because P_1P_2=1$]

case(i):- consider the values of $m_1=0$, $m_2=0$

substituting these values in expression below,

$(P_2^2+P_2)\{P_1^{2m_1}(P_2^2+1)\}^{m_2}P_1^{2m_1}$

$=(P_2^2+P_2)(P_1^{2m_1}(P_2^2+1))^{m_2}P_1^{2m_1}$

$=(P_2^2+P_2)(P_1^{2m_1}(P_2^2+1)\}^{0}P_1^{2(0)}$

$=(P_2^2+P_2)(1\cdot1)\cdot1$ [$\because P^0=1$]

$=P_2^2+P_2$

case(ii): consider the value of $m_1=0$, $m_2=1$

substituting these values in the expression below

$(P_2^2+P_2)(P_1^{2m_1}(P_2^2+1)\}^{m_2}P_1^{2m_1}$

$=(P_2^2+P_2)\{P_1^{2(0)}(P_2^2+1)\}^{1}P_1^{2(0)}$

$=(P_2^2+P_2)(P_2^2+1)$

$=P_2^4+P_2^2+P_2^3+P_2$

$=P_2+P_2^2+P_2^3+P_2^4$

case(iii): $m_1=0$, $m_2=2$

$(P_2^2+P_2)(P_1^{2m_1}(P_2^2+1)\}^{m}P_1^{2m_1}$

$=(P_2^2+P_2)\{P_1^{2(0)}(P_2^2+1)\}^{2}P_1^{2(0)}$

$=(P_2^2+P_2)\{1(P_2^2+1)\}^{2}1$

$=(P_2^2+P_2)\{P_2^4+2P_2^2+1\}$

$=P_2^6+2P_2^4+P_2^2+P_2^5+2P_2^3+P_2$

[$\because$ The looping term is not multiplied with non-loopering term]

$=P_2^6+P_2^4+P_2^5+P_2^3+P_2^2+P_2$

$\Rightarrow\sum_{i=1}^{6}P_2^i$

case(n): consider the value of $m_1=0$ $m_2=2$

substituting these values in the expression below.

$(P_2^2+P_2)(P_1^{2m_1}(P_2^2+1)\}^{m_2}P_1^{2m_1}$

$=(P_2^2+P_2)\{P_1^{2(0)}(P_2^2+1)\}^{2}P_1^{2(0)}$

$=(P_2^2+P_2)(1(P_2^2+1))^{2}1$

$=(P_2^2+P_2)(P_2^4+2P_2^2+1)$

$=P_2^4+2P_2^4+P_2^5+P_2^3+P_2^2+P_2$

For different combinations of m_1, m_2 values of the following Table 5.8 is obtained

Table 5.8 Results of the Push/Pop graph Analysis

m_1	0	0	0	0	1	1	1	1	2	2	2
m_2	0	1	2	3	0	1	2	3	0	1	2
Push/ pop	$a_2^1 + a_2^2$	$a_2^1 + a_2^2 + a_2^3$	a_2^1 new	a_2^1 pop	a_2^2	a_2^2 new	a_2^2 new	a_2^3 new	$a_2^1 + a_2^3$	a_2^3 new	a_2^3 new

GET/RETURN PROBLEM

Get/Return is a complimentary action or operation, which is same as push/pop operation.

There are Arithmetic tables for Get/return similar to Push/pop. The tables are

Table 5.9 Multiplication Table for Get/Return

·	G	R	1
G	G^2	1	G
R	1	R^2	R
1	G	R	1

In the above table,
- G is used for Get
- R is used for Return
- 1 is used for No operation

Table 5.10 Addition Table for Get/Return

+	G	R	1
G	G	G+R	G+1
R	G+R	R	R+1
1	G+1	R+1	1

Example :-

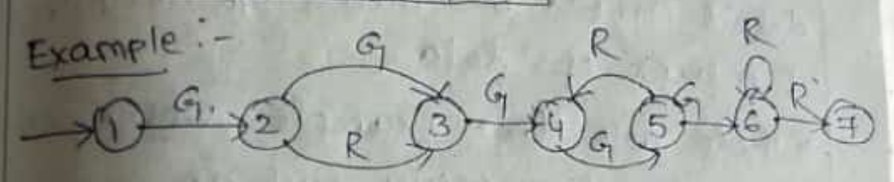


Fig 5.36 Flowgraph for GET/RETURN

By considering the parallel, series and loop rules, the path expreses flowgraph in Fig 5.36 would be

$G_1(G_1+R)G_1(G_1R)^*GG_1R\,{}^*R$

$= G_1(G_1+R)G^3R\,{}^*R \quad [\because G_1R=1]$

$= G_1(G_1+R)G^3R\,{}^*R$

$= (G_1+R)G^3R\,{}^*(G_1R)$

$= (G_1+R)G^3R\,{}^*(1)$

$= (G_1^4 + G^2 \cdot G_1R)R^*$

$= (G_1^4 + G_1^2)R^* \quad [\because G_1R=1]$

The resultant path expression would be

$(G_1^4 + G_1^2)R^*$

Regular Expressions and flow Anomaly detection

→ Data flow Anomaly is the thing where we expect one thing to happen and some other things happens instead

→ Flow Anomaly detection is used to detect errors or flaws in sequence of actions of data.

→ There are few examples where data flow anomalies occur such as,

operations on file : read-r, write-w, open-o, close-c

operations on Tape-Transport :- fast forward-f, rewind-d, skip-s, read-r, write-w, stop-P.

Data flow Anomalies :- defined-d, killed-k, used-u, defining and killed-dk, killing the killing object - kk, etc.,

Logic Based Testing

Overview

→ Logic is the word which is used mostly by the programmers

→ The boolean Algebra is said to be logic in its simplest form.

→ The testing that is performed based on this boolean algebra is known as "Logic Based Testing"

The steps that are to be considered are
- Hardware Logic Testing.
- Specification systems
- Knowledge Based systems

Hardware Logic Testing

This is a primary tool for designers. The automation of hardware testing is done 10-15 years before than automation of software testing.

Specification Systems :-

The major problem with the specifications is that they could be hard to express. To understand specifications or rejection of specifications, the boolean algebra should be known.

Knowledge Based Systems

These are used to construct many applications which are very difficult. These systems will gather ~~information~~ knowledge from a known repository or a domain into a database.

Decision Tables

The table which consists of set of conditions and corresponding actions is known as decision table. The table consists of four areas known as

1. Condition stub
2. Condition Entry
3. Action stub
4. Action Entry

Table 6.1 Decision Table

Condition stub	Condition Entry
Action stub	Action Entry

condition stub:-

The list of names of conditions is known as condition stub. Every column in the table specifies some rules. for these rules to be matched, conditions are marked with yes/No conditions. "yes" specifies the condition should meet the particular rule.

"NO" specifies the condition need not to meet its rule

Condition Entry:-

All the rules are the columns of decision table. The condition for the rules ie, the rule has to be satisfied by the condition or not.

Action Entry

The Action entry contains YES/NO wheather the actions are performed or not based on conditions and rules.

Action stub

As Condition stub consists of list of conditions, Action stub contains list of actions.

Applications of Decision Tables

→ Decision table processors.
→ Decision table for test case design
→ Expansion of immaterial cases
→ Decision tables and structure.

Decision table processors

→ The decision tables are mostly used in business data processing. Hence they are known as decision table processors.

→ The translator will automatically convert the decision tables into source code as they are in high level language.
→ The translator would check for the rule 1 first.
→ If rule 1 is satisfied, the corresponding action to rule 1 would be performed.
→ The process of checking the rules is done until all the rules in the decision table are checked

Advantages
- Integrity of clarity
- Explict relation to specification
- High level maintainability.

Disadvantages
- Insufficient program provides.

Decision tables for test case design
→ The decision table should be used for test case design when decision table acts as specification.
→ Decision tables are used as basis for designing test cases when logic of program is implemented as decision tables
→ Even, if the decision tables are given as programs logic, it will not be possible to implement it.
→ This is because of logic behaviour i.e., it may contain interface restrictions or it may not have sufficient features

Expansion of immaterial cases
→ The immaterial entries in the decision table in which the condition does not play any role.
→ It means the condition does not affect the action wheather it is either "true" or "false".
→ It is denoted by "I" in decision table.
→ The expansion of immaterial cases is done by replacing each "I" entry in table with a pair of cases or entries.

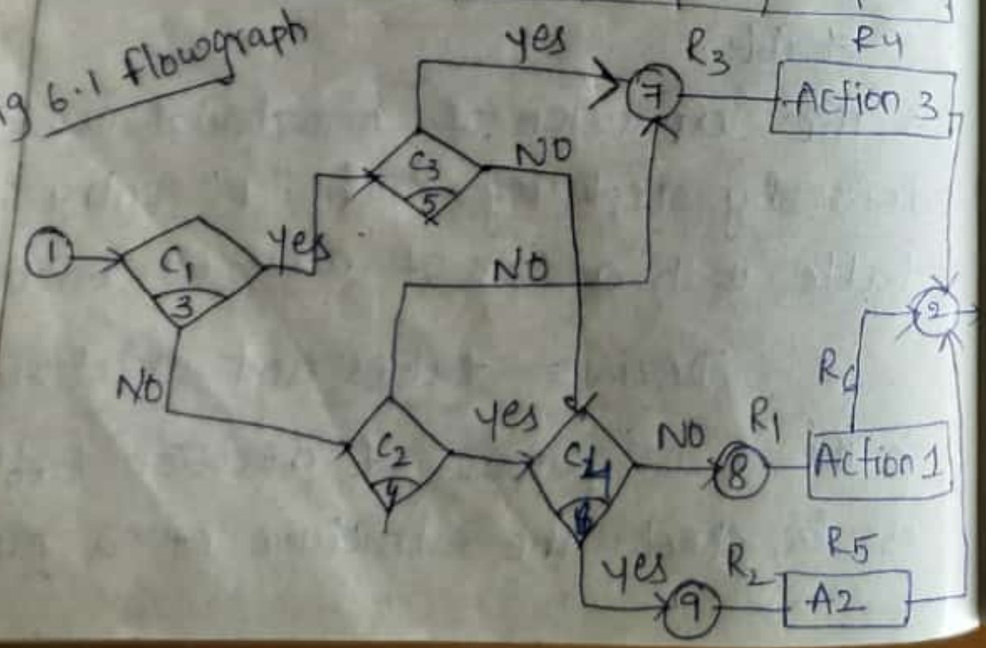
Decision tables and structure
→ The main purpose of decision table is to check the structure of a program

→ It can be represented in the form of a decision tree where, combinations of conditions can lead to executing different actions in order to give expression.

Table 6.7 Decision Table

	R_1	R_{12}	R_3	R_4	R_5	R_6
condition C_1	Yes	No	Yes	No	Yes	No
condition C_2	I	Yes	I	No	I	Yes
condition C_3	No	I	Yes	I	NO	I
condition C_4	No	Yes	I	I	Yes	No
Action A_1	Yes	NO	NO	NO	NO	Yes
Action A_2	No	Yes	NO	NO	Yes	NO
Action A_3	No	NO	Yes	Yes	NO	NO

Fig 6.1 Flowgraph



path Expressions

path expressions can be depends on the expressions which comes from the following ways:

→ Predicates and binary truth values
→ Boolean Algebra
→ Reducing boolean expressions

Predicates and Binary Truth values

Predicate:- The process where the outcome is a truth value is known as predicate.

→ The predicate is based on relational operators includes >, <, =, <=, >=, etc.,

Binary Truth Values:-

Predicates are not restricted to binary with values i.e., true/false such as

1. Multiway logic.
2. Multivalued logic.

Boolean Algebra

- The boolean algebra deals with true and false values of the predicates.
- The true and false values of the Predicates sometimes indicates as "1" and "0

Bodean algebra mainly uses three operators:

1. X- And / Multiplication.
2. + - OR / Addition
3. A- Negation

Table 6.10 Truth Table for AXB

A	B	AXB
T	T	T
T	F	F
F	T	F
F	F	F

Table 6.11 Truth Table for A+B

A	B	A + B
T	T	T
T	F	T
F	T	T
F	F	F

Table 6.12 Truth Table for $\overline{A}$

A	$\overline{A}$
T	F
F	T

i) Associative law of ~~Addition~~:
$(A+B)+C = A+(B+C)$

Table 6.13 Truth Table for associative law of addition

A	B	C	A+B	(A+B)+C
True	True	True	True	True
True	True	False	True	True
True	False	True	True	True
True	False	False	True	True
False	True	True	True	True
False	True	False	True	True
False	False	True	False	True
False	False	False	False	False

(a)

A	B	C	B+C	A+(B+C)
True	True	True	True	True
True	True	False	True	True
True	False	True	True	True
True	False	False	False	True
False	True	True	True	True
False	True	False	True	True
False	False	True	True	True
False	False	False	False	False

(b)

Distributive law

$A + (BC) = (A+B) \times (A+C)$

Table 6.17 Truth Tables for distributive law

A	B	C	BC	A+(BC)
True	True	True	True	True
True	True	False	False	True
True	False	True	False	True
True	False	False	False	True
False	True	True	True	True
False	True	False	False	False
False	False	True	False	False
False	False	False	False	False

(a)

A	B	C	A+B	A+C	(A+B)·(A+C)
True	True	True	True	True	True
True	True	False	True	True	True
True	False	True	True	True	True
True	False	False	True	True	True
False	True	True	True	True	True
False	True	False	True	False	False
False	False	True	False	True	False
False	False	False	False	False	False

(b)

Reducing Boolean Expressions

To reduce the boolean expression, the procedure is,

1. Remove the parenthesis by multiplying all the necessary variables.
2. If any identical terms are there, replace all the identical terms by one term.

$A\bar{B}C + B + B\bar{D} + \bar{A}C$

$= A\bar{B}C + B(1 + \bar{D} + A\bar{D}) + \bar{A}C$

$= A\bar{B}C + B + \bar{A}C$

$= C[A\bar{B} + \bar{A}] + B$

$= C[\bar{B} + \bar{A}] + B$

$= \bar{B}C + \bar{A}C + B$

$= C + \bar{A}C + B$

$= C(1 + \bar{A}) + B$

$= C + B$

$= B + C$

[$\cdot B + \bar{B}C = (B + \bar{B})(B + C) = B + C$]

KV charts

→ The specifications are coasted into sentences by using the following IF-THEN formal

→ if a predicate THEN action.

→ KV (Karnaugh-veitech) chart is known

by every combination with any one map, chart, diagram.

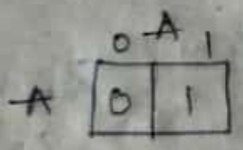
→ It is used to reduces boolean algebraic manipulations to graphical trivia.

Single Variable

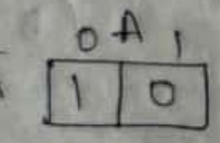
It shows all the boolean functions of a single variable and their equivalent representation as a KV chart.



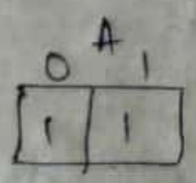
The function is never true.



The function is true when A is true



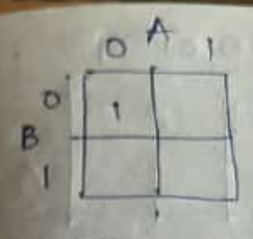
The function is true when A is false



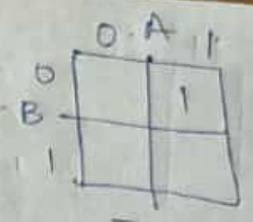
The function is always true.

Two variable

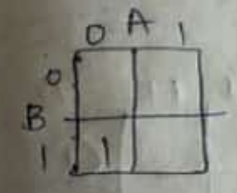
The below diagram shows the 8 of the 16 possible functions of two variables



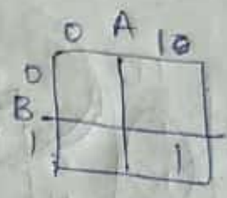
$A\bar{B}$ – NAND



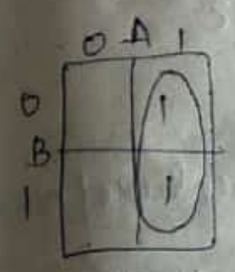
$A\bar{B}$ – A and not B

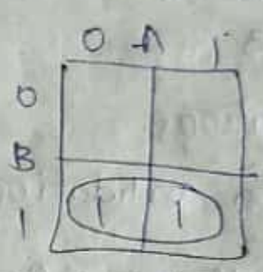


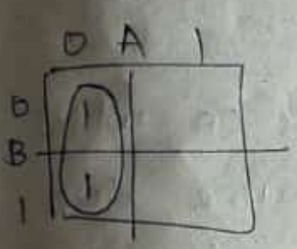
$\bar{A}B$ – B and not A

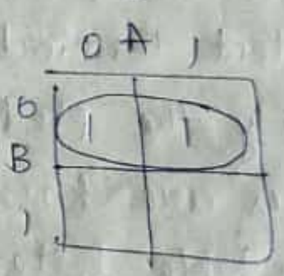


AB – A and B



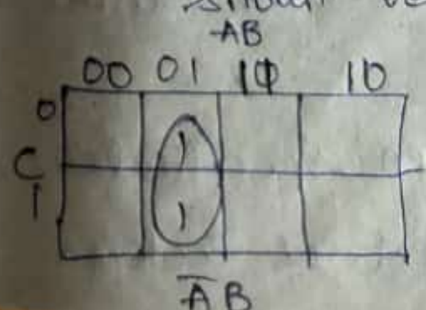


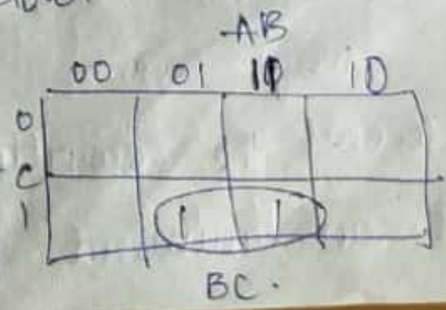


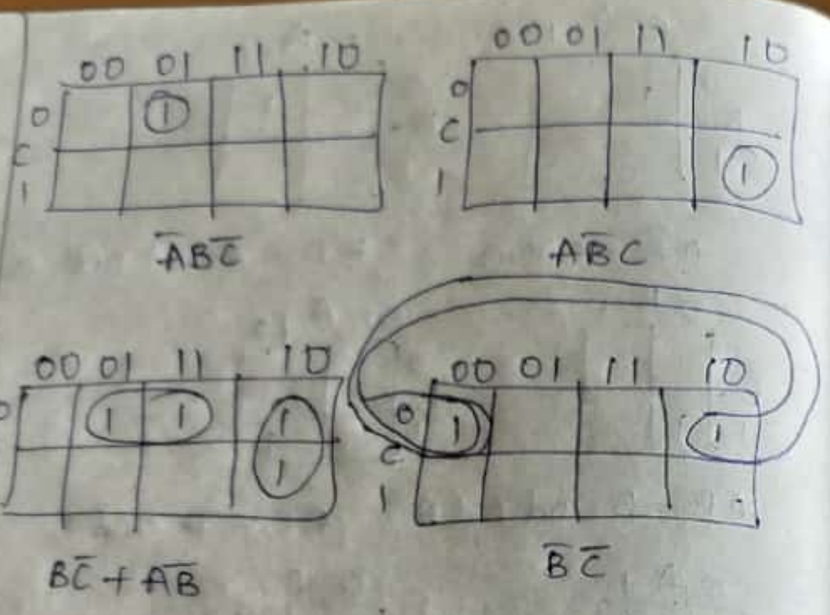


Three Variable

KV charts for three variables are shown below.







Specifications

Specification validation procedure/steps

1. Rewrite the specifications using consistent terminology
2. Identify the predicates on which the cases are based. Name them with suitable letters, such as A, B, C.
3. Rewrite the specification in English that uses only the logical connectives AND, OR and NOT, however, stilted it may seem. ||
4. Convert the rewritten specifications into an equivalent set of Boolean expressions.
5) Identify the default action and cases, if any are specified.
6) Enter the boolean expressions in a KV chart and check for consistency. If the specification are consistent, there will be no overlaps, except for the cases that result in multiple actions.
7) Enter the default cases and check for consistency.
8) If all boxes are covered, the specification is complete.
9) If the specification is incomplete & inconsistent, translate the corresponding boxes of the KV chart back into English and get a clarification, explanation or revision.
10. If the default cases were not specified explicitly, translate the default cases back into English and get a confirmation. ||